

```

//código para fazer autenticação de login e senha

import bcrypt

import sqlite3

import sys

from pathlib import Path

#Adiciona o diretório raiz ao path para garantir as importações

BASE_DIR = Path(__file__).resolve().parent.parent if

str(BASE_DIR) not in sys.path: sys.path.append(str(BASE_DIR))

from database.connection import get_connection

class Session:

    """Guarda os dados do usuário atualmente autenticado no sistema."""

    usuario_id = None

    nome = None

    usuario = None

    cargo = None

    @classmethod

    def set_user(cls, row):

        cls.usuario_id = row["id"]

        cls.nome = row["nome"]

        cls.usuario = row["usuario"]

        cls.cargo = row["cargo"]

    @classmethod

    def clear(cls):

        cls.usuario_id = None

```

cls.nome None

cls.usuario = None

cls.cargo None

@classmethod

def is_admin(cls):

return cls.cargo == "Administrador"

def gerar_hash_senha (senha_texto_puro: str) -> str:

"""Gera um hash bcrypt seguro a partir de uma senha em texto puro.

salt bcrypt.gensalt (rounds=12)

hash_bytes = bcrypt.hashpw(senha_texto_puro.encode('utf-8'), salt)

return hash_bytes.decode('utf-8')

def verificar_senha(senha_texto_puro: str, senha_hash: str) -> bool:

"""Verifica se a senha digitada corresponde ao hash salvo no banco."""

return bcrypt.checkpw(senha_texto_puro.encode('utf-8'),

senha_hash.encode('utf-8'))

def criar_usuario_admin_padrao():

"""Cria o administrador inicial ('admin' / 'admin123') se o banco estiver

vazio.

conn.get_connection() cursor = conn.cursor()

cursor.execute("SELECT COUNT(*) as total FROM usuarios;")

total_usuarios cursor.fetchone() ["total"]

if total_usuarios == 0:

senha_hash gerar_hash_senha("admin123")

cursor.execute(

```

INSERT INTO Usuarios (nome, usuario, senha_hash, cargo, ativo) VALUES (?, ?, ?, ?, 1);

) ("Administrador Padrão", "admin", senha_hash, "Administrador")

conn.commit()

print("[AUTH] Administrador padrão criado com sucesso!")

print(" Login: admin | Senha: admin123")

conn.close()

def autenticar_usuario(usuario login: str, senha plana: str): """Valida as credenciais do
usuário."""

conn = get_connection()

cursor = conn.cursor()

cursor.execute(

"SELECT FROM usuarios WHERE usuario? AND ativo = 1;",

(usuario_login,)

) usuario_row = cursor.fetchone()

conn.close()

if not usuario_row:

return False, "Usuário não encontrado ou inativo."

if verificar_senha (senha_plana, usuario_row["senha_hash"]): Session.set_user
(usuario_row)

return True, f"Bem-vindo, (usuario_row['nome'])!"

return False, "Senha incorreta."

```